SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**
Question Count: 5

Code of Conduct

I G.Piyusha(192472377) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: G.Piyusha

Assessment Tasks Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments.
   Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy.
   Design a solution using cloud-native backup, replication, and versioning.
   Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved. Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.
   Explain how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured. Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

Question 1: Video Streaming Platform Migration

> **Scenario:** *A media company is migrating its video streaming platform to the cloud. It must serve millions of concurrent users with minimal buffering.*

(a) Cloud architecture using scalability and caching

Key issues identified: unpredictable traffic spikes (premieres, live events), very high egress bandwidth, startup delay and buffering for geographically spread viewers, and the need to store and transcode large video libraries.

Viewer Devices	DNS + CDN Edge Cache	Global Load Balancer	Auto-Scaled App / API Tier	Cache + DB + Object Storage

- **Ingest and transcoding:** uploaded masters go to object storage (e.g., Amazon S3 / Azure Blob). Event-triggered, containerised transcoding workers convert each title into multiple bitrates and resolutions and package them as HLS/DASH segments (adaptive bitrate streaming).
- **Storage tier:** durable object storage holds all renditions; hot titles stay in standard tier, older titles move to infrequent-access tiers.
- **Caching (multi-layer):** (1) CDN edge caches serve video segments close to users, giving the biggest reduction in buffering and origin load; (2) a regional origin-shield cache collapses repeated requests; (3) an in-memory cache (Redis/Memcached) holds sessions, catalogue metadata and recommendations; (4) browser/player caching and pre-fetching of the next few segments.
- **Application tier:** stateless microservices (authentication, catalogue, playback, recommendations, billing) in containers behind load balancers, so any instance can serve any request and the tier can scale horizontally.
- **Data tier:** managed relational DB with read replicas for accounts/billing, and a NoSQL store (DynamoDB/Cassandra style) for watch history and viewing progress, which has very high write rates.

(b) How load balancing and auto-scaling ensure smooth streaming

- **Load balancing:** a global DNS/anycast layer routes users to the nearest healthy region; regional Layer-7 load balancers distribute API and manifest requests using least-connections or round-robin, run health checks, and remove failing instances automatically. TLS termination at the balancer reduces CPU load on servers.
- **Auto-scaling:** scaling groups add or remove instances/containers based on metrics such as CPU, request count per target, concurrent sessions and queue depth. Target-tracking policies handle normal variation, scheduled scaling pre-warms capacity before known events (a season launch or live match), and predictive scaling uses historic patterns.
- **Result:** the CDN absorbs most video traffic, while balancer plus auto-scaling keeps the control plane (login, playback start) responsive. Together they keep start-up time low and avoid rebuffering during surges. Graceful degradation (e.g., lowering default bitrate, serving cached catalogue) protects playback if a backend component is stressed.

(c) Cost optimisation strategies

Strategy	How it reduces cost
Maximise CDN cache-hit ratio	Longer TTLs for segments, origin shield and cache-key tuning cut expensive origin egress and compute.
Right-sizing and mixed pricing	Reserved/Savings Plans for the steady baseline, on-demand for spikes, Spot/preemptible instances for fault-tolerant transcoding jobs.
Storage tiering and lifecycle rules	Move rarely watched titles to infrequent-access or archive storage; delete unused intermediate renditions.
Efficient encoding	Per-title encoding and modern codecs (HEVC/AV1) lower bitrate at equal quality,

	reducing storage and bandwidth.
Autoscale in, not just out	Aggressive scale-in policies and scheduled scale-down at off-peak hours avoid paying for idle capacity.
Negotiated CDN/egress contracts	Committed-volume pricing and multi-CDN selection by cost and performance.
Monitoring and budgets	Tagging, cost dashboards, budget alerts and anomaly detection to catch waste early.

Question 2: Containers across Multiple Cloud Providers (Fintech)

Scenario: *A fintech startup deploys containerised applications across several cloud providers and struggles with consistency and deployment management.*

(a) Solution using container orchestration and multi-cloud

Issues identified: configuration drift between providers, different native tooling and APIs, inconsistent security policies, manual and error-prone releases, and difficulty in tracking where each workload runs. For a fintech, regulatory exposure and downtime make these especially serious.

- **Standardise on Kubernetes:** run a managed Kubernetes service on each provider (EKS, AKS, GKE) so that the same container images and manifests work everywhere. A single management plane (e.g., Rancher, Anthos, Azure Arc or Argo CD with cluster registration) provides one view of all clusters.
- **Immutable images and registry:** build once in CI, scan for vulnerabilities, sign the image and replicate it to a registry in each cloud.
- **GitOps and Infrastructure as Code:** Terraform defines clusters, networks and IAM; Helm/Kustomize describe applications; Argo CD or Flux continuously reconciles the cluster state with Git. This removes drift and gives auditable, repeatable deployments with easy rollback.
- **Progressive delivery:** blue-green or canary releases through the CI/CD pipeline, with automated rollback on failed health checks or SLO breach.
- **Policy and security:** Open Policy Agent/Kyverno enforce the same admission rules (no root containers, required resource limits, approved registries); secrets in Vault or cloud KMS; network policies and mTLS between services.
- **Traffic placement:** a global traffic manager (DNS or global load balancer) directs users to the closest or healthiest cluster, and keeps regulated data in the proper region.

(b) Service discovery and scaling

- **Service discovery:** inside each cluster, Kubernetes Services and CoreDNS give stable names to changing pod IPs. Across clusters and clouds, a service mesh (Istio, Linkerd or Consul) with multi-cluster federation provides cross-cluster discovery, load balancing, retries, mTLS and observability without changing application code. External traffic enters via Ingress/API gateway.
- **Scaling:** Horizontal Pod Autoscaler scales pods on CPU, memory or custom metrics (requests per second, queue length); KEDA scales on event sources; Cluster Autoscaler/Karpenter adds or removes nodes; Vertical Pod Autoscaler tunes requests and limits. Resource quotas and pod-disruption budgets keep scaling safe and predictable.

(c) Benefits and risks of multi-cloud

Benefits	Risks / challenges
Avoids vendor lock-in and improves bargaining power	Higher operational complexity and need for broader skills
Resilience: an outage at one provider need not stop the service	Inconsistent IAM, networking and logging across providers
Best-of-breed services and regional coverage for data-residency rules	Inter-cloud data transfer (egress) costs and added latency

Flexibility to move workloads for cost or performance	Larger attack surface and more difficult compliance auditing
Supports disaster-recovery across providers	Lowest-common-denominator design may leave out provider-specific features

Question 3: Backup and Disaster Recovery (E-learning Platform)

Scenario: *An e-learning platform lost data through accidental deletion in the cloud and needs a robust backup and disaster-recovery (DR) strategy.*

(a) Solution using backup, replication and versioning

Issues identified: no protection against human error, a single copy of data, no point-in-time recovery, and over-broad delete permissions. Replication alone is not a backup, because a deletion is replicated too; versioning and immutable backups are needed.

Primary DB + Object Storage	Versioning + Soft Delete	Scheduled Snapshots / Backups	Cross-Region Replica	Immutable Vault (Object Lock)

- **Versioning:** enable object versioning and MFA-delete on the buckets holding course videos, assignments and certificates, so deleted or overwritten objects can be restored. Lifecycle rules expire old versions after a set period to limit cost.
- **Cloud-native backup:** AWS Backup / Azure Backup / GCP Backup and DR with policy-based schedules (e.g., hourly incremental, daily full) for databases, volumes and file shares. Enable point-in-time recovery (continuous transaction-log backup) for the student and grade database.
- **Replication:** asynchronous cross-region replication of storage and database read replicas for regional failure; synchronous multi-AZ deployment for the primary database for local failures.
- **Protection from deletion:** immutable (WORM) backup vault/object lock, a separate backup account with restricted access, least-privilege IAM, approval workflow for destructive actions, and encrypted backups.
- **Testing:** automated restore tests each month, with results logged, since an untested backup is not a guarantee.

(b) Achieving RTO and RPO

RPO (Recovery Point Objective) is the maximum acceptable data loss, measured in time. **RTO (Recovery Time Objective)** is the maximum acceptable time to restore service.

Data / service	RPO – how achieved	RTO – how achieved
Student, grade and payment DB (critical)	Near-zero to 5 minutes: continuous log backup, multi-AZ synchronous standby, cross-region replica	15–60 minutes: automated failover to standby, pre-provisioned infrastructure (warm standby), scripted runbooks
Course content in object storage	Minutes: versioning plus cross-region replication	Under 1 hour: point the application at the replica bucket or restore a prior version
Application tier	Not applicable (stateless; images in registry, config in Git)	Minutes to 1 hour: redeploy with IaC and auto-scaling in DR region
Logs and analytics (non-critical)	24 hours: daily backup	24 hours: restore on demand (cold/backup-restore tier)

Shorter RPO is obtained through more frequent backups or continuous replication; shorter RTO through warm or hot standby, automation and rehearsal. Both targets are set according to business impact and cost.

(c) Steps to ensure business continuity during failures

1. Perform a business impact analysis and define RTO/RPO per service; document a DR plan with clear roles and an escalation chain.

2. Detect the failure through health checks and alarms, then declare an incident and notify the response team and stakeholders.
3. Contain the problem: stop further deletion or corruption by revoking affected credentials and freezing writes where necessary.
4. Fail over: promote the replica or standby database, switch DNS/traffic manager to the healthy region, and scale the application tier there.
5. Restore lost data from versions or point-in-time backups and validate integrity with checksums and sample checks.
6. Keep learners informed with a status page and enable degraded mode (e.g., read-only course access, cached lectures) while recovery continues.
7. Fail back once the primary region is stable, then conduct a post-incident review and update the runbooks.
8. Run regular DR drills and backup-restore tests, and review the plan after every significant change.

Question 4: Hybrid Cloud for a Government Agency

> **Scenario:** *A government agency adopts a hybrid cloud: sensitive data and certain applications stay on-premises for compliance, while non-sensitive workloads use the public cloud.*

(a) Hybrid architecture with secure communication

Issues identified: data-classification and residency rules, secure and reliable connectivity, a consistent security posture across two environments, and the risk of data leaving the approved boundary.

On-Premises: Sensitive Apps + Databases	Private Link / VPN Gateway	Firewall + Zero-Trust Gateway	Public Cloud VPC: Non-Sensitive Apps

- **Workload placement:** classify data (public, internal, sensitive/classified). Sensitive systems and databases remain in the on-premises data centre or a government-approved private cloud; portals, analytics on anonymised data, development/test and DR run in the public cloud.
- **Connectivity:** a dedicated private link (AWS Direct Connect, Azure ExpressRoute) as the primary path with IPSec site-to-site VPN as an encrypted backup. Networks are segmented using VPCs/VNets, subnets, security groups and a DMZ; traffic between environments is inspected by next-generation firewalls and IDS/IPS.
- **API gateway and zero trust:** applications talk through authenticated, rate-limited API gateways; no direct database exposure; every request is verified (never trust the network location).
- **Consistent platform:** Kubernetes or a hybrid platform (Azure Arc, Anthos, AWS Outposts) gives the same tooling in both places. Centralised logging and a SIEM collect events from both environments.

(b) Identity management and encryption

- **Identity management:** a single identity provider (e.g., Active Directory federated with the cloud directory) with SSO using SAML/OIDC, MFA for all privileged and remote access, role-based/attribute-based access control, least privilege, just-in-time privileged access (PIM) and regular access reviews. Service-to-service access uses short-lived tokens or managed identities rather than static keys. Joiner-mover-leaver processes are automated through lifecycle management.
- **Encryption in transit:** TLS 1.2/1.3 for all application traffic, IPSec on the links, and mutual TLS between services.
- **Encryption at rest:** AES-256 for databases, volumes, object storage and backups.
- **Key management:** keys in an HSM-backed KMS; for the strictest data the agency keeps control of keys on-premises (bring-your-own-key / hold-your-own-key); automatic rotation, separation of duties between key custodians and administrators, and complete audit logs of key use.

- **Additional controls:** data-loss-prevention policies, tokenisation or masking before any data moves to the public cloud.

(c) Challenges in managing hybrid cloud

Challenge	Mitigation
Security and policy inconsistency across environments	Central policy-as-code, unified IAM and SIEM, regular compliance scans
Network latency, bandwidth and link reliability	Redundant private links, caching, careful placement of chatty components
Compliance and auditing of data residency	Data classification, geo-fencing, automated evidence collection
Operational complexity and skills gap	Common tooling (IaC, Kubernetes), training, automation
Visibility and monitoring across both sides	Unified observability platform and dashboards
Integration of legacy on-premises systems	API wrappers, gradual modernisation, integration middleware
Cost tracking and data-egress charges	Tagging, chargeback reporting, minimise cross-boundary transfers

Question 5: High Availability and Fault Tolerance for a SaaS Provider

Scenario: *A SaaS provider needs the application to continue operating even if one entire cloud region goes down.*

(a) Multi-region deployment strategy

Issues identified: a single region is a single point of failure; failure detection and traffic redirection must be automatic; data must be consistent enough after failover; and shared dependencies (DNS, identity, CI/CD) must not be region-bound.

Users	Global DNS / Anycast Traffic Manager	Region A: LB + App + DB (Primary)	Region B: LB + App + DB (Replica)

- **Pattern choice:** an **active-active** deployment (both regions serve traffic) gives the lowest RTO and best latency for global users but requires careful conflict handling. An **active-passive / warm-standby** design is cheaper and simpler, with the secondary scaled down but ready. A common practice is active-active for stateless tiers and primary/replica for the strongly consistent database.
- **Within each region:** deploy across at least three Availability Zones with auto-scaling groups behind a load balancer, so zone failures never require a regional failover.
- **Global routing:** health-checked DNS (Route 53, Azure Traffic Manager) with latency-based or weighted routing, or a global load balancer/anycast front door; low TTLs allow quick redirection.
- **Infrastructure as Code:** identical stacks created by Terraform/CloudFormation in each region, preventing configuration drift. Container images and artefacts are replicated to both regions.
- **Stateless design:** sessions kept in a distributed cache or tokens, so any region can handle any user.

(b) Data replication and failover configuration

- **Relational data:** a managed database with a cross-region replica (e.g., Aurora Global Database, Azure SQL failover group, Cloud Spanner). Replication lag is usually under a second; promote the replica on failover. Synchronous replication gives RPO≈0 but adds latency, so asynchronous replication is typically chosen across regions, accepting small potential loss.
- **NoSQL and multi-master:** use global tables (DynamoDB, Cosmos DB) for active-active writes, with conflict resolution (last-writer-wins or application logic) and idempotent operations.

- **Object storage and files:** cross-region replication with versioning. **Messaging:** replicated or per-region queues with de-duplication.
- **Failover mechanism:** (1) health checks detect regional failure from several vantage points; (2) the traffic manager automatically stops routing to the failed region; (3) the replica database is promoted (automatic for managed services, or via a runbook with approval to avoid split-brain); (4) auto-scaling expands the surviving region; (5) once recovered, the former primary is re-synchronised and traffic returned gradually (failback).
- **Safeguards:** quorum or fencing to prevent split-brain, circuit breakers and retries with exponential backoff in clients, and regular chaos-engineering/game-day tests to validate failover.

(c) Monitoring and alerting for proactive detection

Area	Approach
Metrics	CloudWatch/Azure Monitor/Prometheus: latency, error rate, saturation, traffic (golden signals), replication lag, CPU/memory, queue depth
Logs and traces	Centralised logging (ELK/Cloud Logging) and distributed tracing (OpenTelemetry, X-Ray, Jaeger) to find the root cause quickly
Synthetic monitoring	Probes from several geographies run login and key user journeys continuously, detecting outages before customers report them
SLOs and alerting	Define SLIs/SLOs and error budgets; alert on burn rate, not just thresholds, to reduce noise; use anomaly detection
Notification and escalation	PagerDuty/Opsgenie/SNS with severity levels, on-call rotation and runbook links; ChatOps notifications
Dashboards and status page	Per-region health dashboards for operations and a public status page for customers
Review	Post-incident reviews feed back into alerts, capacity planning and automation

Conclusion

Across the five scenarios, the common design principles are: scale horizontally and keep services stateless, cache close to the user, automate deployments and recovery with Infrastructure as Code, layer security (identity, encryption, network segmentation), replicate and back up across failure domains, and monitor continuously so that problems are found before users are affected. Each decision balances performance, availability, compliance and cost.